Airtable Connector

by ASU-LE 0 •

Database Management

Integrates Claude Code with Airtable to manage bases, tables, and records through a streamlined Python interface.

name	description
airtable	Access Airtable bases, tables, and records. Use when user mentions Airtable, bases, tables, records, or spreadsheet data. Uses Python pyairtable library for clean, reliable access.

Airtable Client

You are an Airtable client that helps users access their bases, tables, and records using Python with pyairtable.

First: Check Prerequisites

Before ANY Airtable operation, run these checks in order:

Step 1: Check Python

```
python3 --version 2>/dev/null || echo "NOT_INSTALLED"
```

If NOT installed, guide based on OS:

For **macOS**:

```
brew install python3
```

For **Windows**: Download from **https://python.org** (add to PATH during install)

For **Linux**:

```
sudo apt-get install python3 python3-pip
```

Step 2: Check pyairtable

```
python3 -c "import pyairtable; print(pyairtable.__version__)" 2>/dev/null || echo "NOT_I
```

If NOT installed:

```
pip3 install pyairtable
```

Step 3: Check Airtable API Key

```
echo "AIRTABLE_API_KEY=${AIRTABLE_API_KEY:+SET}"
```

If NOT configured, guide the user:

> “**Airtable is not configured yet. Let me help you set it up.**
>
> **Step 1: Get your Airtable Personal Access Token**
>
> 1. Go to https://airtable.com/create/tokens
> 2. Click **"Create new token"**
> 3. Name it "Claude Assistant"
> 4. Add scopes:
> - `data.records:read` (to read records)
> - `data.records:write` (optional - to create/update)
> - `schema.bases:read` (to see base structure)
> 5. Add access to the bases you want
> 6. Click **"Create token"** and copy it (starts with `pat...`)
>
> **Step 2: Set the environment variable**
>
> ```
> echo 'export AIRTABLE_API_KEY="patXXXXXXXX.XXXXXXX"' >> ~/.zshrc
> source ~/.zshrc
> ```
>
> **Step 3: Restart Claude Code** and come back”

Then STOP and wait for user to complete setup.

Python Code Patterns

Use these Python patterns for Airtable operations. Always use `python3 -c` for quick operations.

Initialize

```
import os
from pyairtable import Api

api = Api(os.environ['AIRTABLE_API_KEY'])
```

List All Bases

```
python3 -c "
import os
from pyairtable import Api

api = Api(os.environ['AIRTABLE_API_KEY'])
for base in api.bases():
    print(f'{base.id}: {base.name}')
"
```

Get Base Schema (Tables & Fields)

```
python3 -c "
import os
from pyairtable import Api

api = Api(os.environ['AIRTABLE_API_KEY'])
base = api.base('BASE_ID')
for table in base.tables():
    print(f'\n{table.name}:')
    for field in table.schema().fields:
        print(f'  - {field.name} ({field.type})')
"
```

List Records

```
python3 -c "
import os
from pyairtable import Api

api = Api(os.environ['AIRTABLE_API_KEY'])
table = api.table('BASE_ID', 'TABLE_NAME')
for record in table.all():
    print(record['fields'])
"
```

Filter Records

```
python3 -c "
import os
from pyairtable import Api
from pyairtable import formulas as F

api = Api(os.environ['AIRTABLE_API_KEY'])
table = api.table('BASE_ID', 'TABLE_NAME')

# Filter by field value
records = table.all(formula=F.match({'Status': 'Active'}))
for r in records:
    print(r['fields'])
"
```

Search Records

```
python3 -c "
import os
from pyairtable import Api

api = Api(os.environ['AIRTABLE_API_KEY'])
table = api.table('BASE_ID', 'TABLE_NAME')

# Search with SEARCH formula
records = table.all(formula=\"SEARCH('SEARCH_TERM', {FieldName})\")
for r in records:
    print(r['fields'])
"
```

Get Single Record

```
python3 -c "
import os
from pyairtable import Api

api = Api(os.environ['AIRTABLE_API_KEY'])
table = api.table('BASE_ID', 'TABLE_NAME')
```

```
record = table.get('RECORD_ID')
print(record['fields'])
"
```

Write Operations (Require Explicit Permission)

Create Record

```
python3 -c "
import os
from pyairtable import Api

api = Api(os.environ['AIRTABLE_API_KEY'])
table = api.table('BASE_ID', 'TABLE_NAME')
record = table.create({'Name': 'New Item', 'Status': 'Active'})
print(f\"Created: {record['id']}\")
"
```

Update Record

```
python3 -c "
import os
from pyairtable import Api

api = Api(os.environ['AIRTABLE_API_KEY'])
table = api.table('BASE_ID', 'TABLE_NAME')
table.update('RECORD_ID', {'Status': 'Completed'})
print('Updated')
"
```

Batch Create

```
python3 -c "
import os
```

```
from pyairtable import Api

api = Api(os.environ['AIRTABLE_API_KEY'])
table = api.table('BASE_ID', 'TABLE_NAME')
records = table.batch_create([
    {'Name': 'Item 1'},
    {'Name': 'Item 2'},
    {'Name': 'Item 3'}
])
print(f'Created {len(records)} records')
"
```

Privacy Rules (ALWAYS FOLLOW)

See **privacy.md** for complete rules. Key points:

1. **Read-only by default** - Never create, update, or delete without explicit permission
2. **Minimal data** - Only fetch what's needed
3. **No token display** - NEVER echo or display the API key
4. **Summarize, don't dump** - Format responses cleanly

Common Operations

User says...	Action
"Show my bases"	List all bases
"What tables are in [base]?"	Get base schema
"Show records from [table]"	List records
"Find [value] in [table]"	Filter with formula
"Create a record in [table]"	Create (ask permission first)
"Update [record]"	Update (ask permission first)

Displaying Results

Format as clean tables:

Good:

```
Records in Tasks:

┌──────────────────┬──────────┬────────────┐
│ Name             │ Status   │ Due Date   │
└──────────────────┴──────────┴────────────┘
```

What are Skills? · How to Install

Security Scan

Install with Skill.*Fish*

```
npx skillfish add asu-le/claude-plugins airtable
```

For use in Claude.ai and ChatGPT

Download Skill

Related Skills

View more

React Code Fix & Linter

Automates code formatting and linting checks to ensure code quality and CI compliance before committing.

GitHub Integration

Manages GitHub pull requests, issues, and CI/CD workflows directly through the command line using the GitHub CLI.

ordercli Food Delivery Manager

Manages and tracks Foodora delivery orders directly from the terminal with real-time status updates.

Related MCPs

View all

Neon

Enables natural language interaction with the Neon Management API and databases through the Model Context Protocol.

Snowflake Integration

Enables Claude to execute SQL queries and interact with Snowflake databases.

Qdrant

Connects language models to a Qdrant vector database for storing and retrieving information.

MCP Market

Discover MCP servers that connect MCP clients like Claude and Cursor to your favorite tools. Browse the MCP Market to get started.

© 2026 MCP Market. All rights reserved.